

Programming Language Constructs Polymorphic Types

Peter Sestoft
Niels Hallenberg
2026-07-24



Plan for today

- Polymorphic types
 - Informal procedure
 - Type rules
 - Unification
 - The union-find data structure
 - Type inference algorithm
- Variant generic types in Java and C#
 - Java use-side variance
 - C# 4.0 declaration-side variance

ML/F#-style parametric polymorphism

```
let f x = 1  
in f 2 + f true
```

Type for f is
'a -> int

int -> int

bool -> int

- Each expression has a compile-time type
- The type may be *polymorphic* ('many forms') and have multiple *type instances*

Type generalization and specialization

- If f has type $(\alpha \rightarrow \text{int})$ and α appears nowhere else, the type gets generalized to a *type scheme* written $\forall\alpha.(\alpha \rightarrow \text{int})$:

```
let f x = 1
```

$\forall\alpha.(\alpha \rightarrow \text{int})$

- If f has type scheme $\forall\alpha.(\alpha \rightarrow \text{int})$ then α may be instantiated by/specialized to any type:

```
f 42
```

$f : \text{int} \rightarrow \text{int}$

```
f false
```

$f : \text{bool} \rightarrow \text{int}$

```
f [22]
```

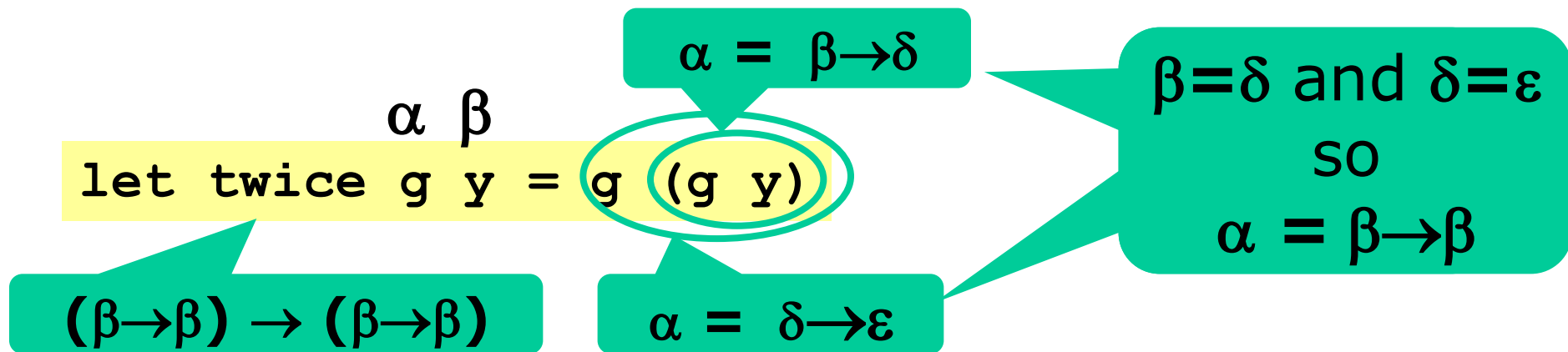
$f : \text{int list} \rightarrow \text{int}$

```
f (3, 4)
```

$f : \text{int} * \text{int} \rightarrow \text{int}$

Polymorphic type inference

- F# and ML have polymorphic type *inference*
- Static types, but not explicit types on functions



- We *generalize* β , so **twice** gets the type scheme $\forall \beta. (\beta \rightarrow \beta) \rightarrow (\beta \rightarrow \beta)$, hence “ β may be any type”

```
let mul2 y = 2 * y
```

mul: int -> int

```
twice mul2 11
```

twice : (int->int)->(int->int)

Basic elements of type inference

- “Guess” types using type variables $\alpha, \beta, \dots$
- Build and solve “type equations” $\alpha = \beta \rightarrow \delta \dots$
- *Generalize* types of let-bound variables/funs. to obtain type schemes $\forall \beta. (\beta \rightarrow \beta) \rightarrow (\beta \rightarrow \beta)$
- *Specialize* type schemes at variable use
- This type system has several names:
 - ML-polymorphism
 - let-polymorphism
 - Hindley-Milner polymorphism (Hindley 1969 & Milner 1978)

Restrictions on ML polymorphism, 1

- Only let-bound variables and functions can have a polymorphic type
- A *parameter's* type is never polymorphic:

```
let f g = g 7 + g false
```

Ill-typed:
parameter g never
polymorphic

- A function is not polymorphic in its own body:

```
let rec h x =  
  if true then 22  
  else h 7 + h false
```

Ill-typed: h not
polymorphic in its
own body

Restrictions on ML polymorphism, 2

- Types must be finite and non-circular

```
let rec f x = f f
```

f not polymorphic
in its own body

- Guess x has type α
- Then f must have type $\alpha \rightarrow \beta$ for some β
- But because we apply f to itself in $(f\ f)$, we must have $\alpha = \alpha \rightarrow \beta$
- But then $\alpha = (\alpha \rightarrow \beta) \rightarrow \beta = ((\alpha \rightarrow \beta) \rightarrow \beta) \rightarrow \beta = \dots$ is not a finite type
- So the example is ill-typed

Restrictions on ML polymorphism, 3

- A type parameter that is used in an enclosing scope cannot be generalized

```
let f x =  $\beta$ 
  let g y = if x=y then 11 else 22
in g false
```

$\alpha = \beta$

$g : \beta \rightarrow \text{int}$

α bound in outer scope, cannot generalize β

Ill-typed: function g not polymorphic

- Reason: If this were well-typed, we would compare x (42) with y (false), not good...

Joint exercises

- Which of these are well-typed, and why/not?

```
let f x = 1  
in f f
```

```
let f g = g g
```

```
let f x =  
    let g y = y  
    in g false  
in f 42
```

```
let f x =  
    let g y = if true then y else x  
    in g false  
in f 42
```

Type rules for ML-polymorphism

$$\frac{}{\rho \vdash i : \text{int}} \quad (p1)$$

$$\frac{}{\rho \vdash b : \text{bool}} \quad (p2)$$

$$\frac{\rho(x) = \forall \alpha_1, \dots, \alpha_n. t}{\rho \vdash x : [t_1/\alpha_1, \dots, t_n/\alpha_n]t} \quad (p3)$$

Specialize from
typescheme

$$\frac{\rho \vdash e_1 : \text{int} \quad \rho \vdash e_2 : \text{int}}{\rho \vdash e_1 + e_2 : \text{int}} \quad (p4)$$

$$\frac{\rho \vdash e_1 : \text{int} \quad \rho \vdash e_2 : \text{int}}{\rho \vdash e_1 < e_2 : \text{bool}} \quad (p5)$$

Generalize to
typescheme

$$\frac{\rho \vdash e_r : t_r \quad \rho[x \mapsto \forall \alpha_1, \dots, \alpha_n. t_r] \vdash e_b : t \quad \alpha_1, \dots, \alpha_n \text{ not free in } t}{\rho \vdash \text{let } x = e_r \text{ in } e_b \text{ end} : t} \quad (p6)$$

$$\frac{\rho \vdash e_1 : \text{bool} \quad \rho \vdash e_2 : t \quad \rho \vdash e_3 : t}{\rho \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : t} \quad (p7)$$

(p8)

$$\frac{\rho[x \mapsto t_x, f \mapsto t_x \rightarrow t_r] \vdash e_r : t_r \quad \rho[f \mapsto \forall \alpha_1, \dots, \alpha_n. t_x \rightarrow t_r] \vdash e_b : t \quad \alpha_1, \dots, \alpha_n \text{ not free in } \rho}{\rho \vdash \text{let } f x = e_r \text{ in } e_b \text{ end} : t} \quad (p8)$$

$$\frac{\rho \vdash e_1 : t_x \rightarrow t_r \quad \rho \vdash e_2 : t_x}{\rho \vdash e_1 e_2 : t_r} \quad (p9)$$

Joint exercises

- Draw the type trees for some of these

```
let x = 1  
in x < 2
```

```
let f x = 1  
in f 2 + f false
```

```
let f x = 1  
in f f
```

Programming type inference

- Algorithm W (Damas & Milner 1982) with many later improvements
- Symbolic type equation solving by
 - Unification
 - The union-find data structure
- “Not free in ρ ” formalized by binding levels:

$\alpha = \beta$

```
0 let f $\alpha:0$  x $\beta:0$  =  
  1 let g $\beta:0$  y = if x=y then 11 else 22  
    in g false  
in f 42
```

- Since β -level < g-level, do not generalize β



Unification of two types, $\text{unify}(t_1, t_2)$

Type t_1	Type t_2	Action
int	int	No action
bool	bool	No action
$t_{1x} \rightarrow t_{1r}$	$t_{2x} \rightarrow t_{2r}$	$\text{unify}(t_{1x}, t_{2x})$ and $\text{unify}(t_{1r}, t_{2r})$
α	α	No action
α	β	Make $\alpha = \beta$
α	t_2	Make $\alpha = t_2$ unless t_2 contains α
t_1	β	Make $\beta = t_1$ unless t_1 contains β
All other cases		Failure, type error!

To avoid circular and infinite type.



The union-find data structure

- A graph of nodes (type variables) divided into disjoint classes
- Each class has a representative node
- Operations:
 - New: create new node (type variable)
 - Find(n): find representative of node n 's class
 - Union(n_1, n_2): join the classes of n_1 and n_2

Type inference for micro-ML, 1

```
let rec typ (lvl : int) (env : tenv) (e : expr) : typ =  
  match e with  
  | CstI i -> TypI  
  | CstB b -> TypB  
  | Var x   -> specialize lvl (lookup env x)  
  | ...
```

$$\frac{}{\rho \vdash i : \text{int}}$$
$$\frac{}{\rho \vdash b : \text{bool}}$$
$$\frac{\rho(x) = \forall \alpha_1, \dots, \alpha_n. t}{\rho \vdash x : [t_1/\alpha_1, \dots, t_n/\alpha_n]t}$$

Replace type
variables in type
scheme for x with
fresh type
variables.

$$\text{typ } \rho \ e = t$$

if and only if

$$\rho \vdash e : t$$

Type inference for micro-ML, 2

```
let rec typ (lvl : int) (env : tenv) (e : expr) : typ =  
  match e with  
  | Prim(ope, e1, e2) ->  
    let t1 = typ lvl env e1  
    let t2 = typ lvl env e2  
    match ope with  
    | "*" -> (unify TypI t1; unify TypI t2; TypI)  
    | "+" -> (unify TypI t1; unify TypI t2; TypI)  
    | "=" -> (unify t1 t2; TypB)  
    | "<" -> (unify TypI t1; unify TypI t2; TypB)  
    | "&" -> (unify TypB t1; unify TypB t2; TypB)  
    | _ -> failwith ("unknown primitive " ^ ope)
```

$$\frac{\rho \vdash e_1 : \text{int} \quad \rho \vdash e_2 : \text{int}}{\rho \vdash e_1 + e_2 : \text{int}}$$
$$\frac{\rho \vdash e_1 : \text{int} \quad \rho \vdash e_2 : \text{int}}{\rho \vdash e_1 < e_2 : \text{bool}}$$

Type inference for micro-ML, 3

```
let rec typ (lvl : int) (env : tenv) (e : expr) : typ =  
  match e with  
  | If(e1, e2, e3) ->  
    let t2 = typ lvl env e2  
    let t3 = typ lvl env e3  
    unify TypB (typ lvl env e1) ;  
    unify t2 t3;  
    t2
```

Can we replace t_2
with t_3 ?

$$\frac{\rho \vdash e_1 : \text{bool} \quad \rho \vdash e_2 : t \quad \rho \vdash e_3 : t}{\rho \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : t}$$

Type inference for micro-ML, 4

```
let rec typ (lvl : int) (env : tenv) (e : expr) : typ =  
  match e with  
  | ...  
  | Let(x, eRhs, letBody) ->  
    let lvl1 = lvl + 1  
    let resTy = typ lvl1 env eRhs  
    let letEnv = (x, generalize lvl resTy) :: env  
    typ lvl letEnv letBody  
  | ...
```

Create type scheme for x in
body of `let`-expression.

$$\frac{\rho \vdash e_r : t_r \quad \rho[x \mapsto \forall \alpha_1, \dots, \alpha_n. t_r] \vdash e_b : t \quad \alpha_1, \dots, \alpha_n \text{ not free in } \rho}{\rho \vdash \text{let } x = e_r \text{ in } e_b \text{ end} : t}$$

Type inference for micro-ML, 5

```
let rec typ (lvl : int) (env : tenv) (e : expr) : typ =  
  match e with  
  | ...  
  | Letfun(f, x, fBody, letBody) ->  
    let lvl1 = lvl + 1  
    let fTyp = TypV(newTypeVar lvl1)  
    let xTyp = TypV(newTypeVar lvl1)  
    let fBodyEnv = (x, TypeScheme([], xTyp))  
                  :: (f, TypeScheme([], fTyp)) :: env  
    let rTyp = typ lvl1 fBodyEnv fBody  
    let _ = unify fTyp (TypF(xTyp, rTyp))  
    let bodyEnv = (f, generalize lvl fTyp) :: env  
    typ lvl bodyEnv letBody  
  | ...
```

Create type scheme for x in
body of `let`-expression.

$$\frac{\rho[x \mapsto t_x, f \mapsto t_x \rightarrow t_r] \vdash e_r : t_r \quad \rho[f \mapsto \forall \alpha_1 \dots \alpha_n. t_x \rightarrow t_r] \vdash e_b : t \quad \alpha_1 \dots \alpha_n \text{ not free in } \rho}{\rho \vdash \text{let } f x = e_r \text{ in } e_b \text{ end} : t} \quad (p8)$$

Type inference for micro-ML, 6

```
let rec typ (lvl : int) (env : tenv) (e : expr) : typ =  
  match e with  
  | ...  
  | Call(eFun, eArg) ->  
    let tf = typ lvl env eFun  
    let tx = typ lvl env eArg  
    let tr = TypV(newTypeVar lvl)  
    unify tf (TypF(tx, tr));  
    tr  
  | ...
```

$$\frac{\rho \vdash e_1 : t_x \rightarrow t_r \quad \rho \vdash e_2 : t_x}{\rho \vdash e_1 e_2 : t_r} (p9)$$

Properties of ML-style polymorphism

- The type found by the inference algorithm is the most general one: the *principal type*
- Consequence: Type checking can be modular
- But types can be large, type inference slow:

```
let id x = x
let pair x y p = p x y
let p1 p = pair id id p
let p2 p = pair p1 p1 p
let p3 p = pair p2 p2 p
let p4 p = pair p3 p3 p;;
let p5 p = pair p4 p4 p;;
```

Exponentially
many type
variables!

- In practice types are small and inference fast



Plan for today

- Polymorphic types
 - Informal procedure
 - Type rules
 - Unification
 - The union-find data structure
 - Type inference algorithm
- **Variant generic types in Java and C#**
 - **Java use-side variance**
 - **C# 4.0 declaration-side variance**

Type inference in C# 3.0

```
var x = "hello";           // Inferred type: String
... x.Length ...
x = 17;                     // Type error
```

- No polymorphic generalization
- Can infer parameter type of anonymous function from context: `xs.Where(x=>x*x>5)`
- Cannot infer type of anonymous function
- Parameter types in methods
 - must be declared
 - cannot be inferred, because C# allows method overloading ...

Polymorphism (generics) in Java and C#

- Polymorphic types

```
interface IEnumerable<T> { ... }  
class List<T> : IEnumerable<T> { ... }  
struct Pair<T,U> { T fst; U snd; ... }  
delegate R Func<A,R>(A x);
```

- Polymorphic methods

```
void Process<T>(Action<T> act, T[] xs)
```

C#

```
void <T> Process(Action<T> act, T[] arr)
```

Java

- Type parameter constraints

```
void Sort<T>(T[] arr) where T : IComparable<T>
```

C#

```
void <T extends Comparable<T>> Sort(T[] arr)
```

Java

Variance in type parameters

- Assume Student subtype of Person

```
void PrintPeople(IEnumerable<Person> ps) { ... }
```

```
IEnumerable<Student> students = ...;  
PrintPeople(students);
```

Java and C# 3 say
NO: Ill-typed!

- C# 3 and Java:
 - A generic type is *invariant* in its parameter
 - I<Student> is *not* subtype of I<Person>
- Co-variance (co=with):
 - I<Student> is subtype of I<Person>
- Contra-variance (contra=against):
 - I<Person> is subtype of I<Student>

Co-/contra-variance is unsafe in general

- Co-variance is unsafe in general

```
List<Student> ss = new List<Student>();  
List<Person> ps = ss;  
ps.Add(new Person(...));  
Student s0 = ss[0];
```

Wrong!

Because would allow
writing Person to
Student list

- Contra-variance is unsafe in general

```
List<Person> ps = ...;  
List<Student> ss = ps;  
Student s0 = ss[0];
```

Wrong!

Because would allow
reading Student from
Person list

- But:

- co-variance OK if we *only read (output)* from list
- contra-variance OK if we *only write (input)* to list

Java 5 wildcards

- Use-side co-variance

```
void PrintPeople(ArrayList<? extends Person> ps) {  
    for (Person p : ps) { ... }  
}  
...  
PrintPeople(new ArrayList<Student>());
```

OK!

- Use-side contra-variance

```
void AddStudentToList(ArrayList<? super Student> ss) {  
    ss.add(new Student());  
}  
...  
AddStudentToList(new ArrayList<Person>());
```

OK!

Co-variance in interfaces (C# 4)

- When an `I<T>` only produces/*outputs* `T`'s, it is safe to use an `I<Student>` where a `I<Person>` is expected
- This is co-variance
- Co-variance is declared with the `out` modifier

```
interface IEnumerable<out T> {  
    IEnumerator<T> GetEnumerator();  
}  
interface IEnumerator<out T> {  
    T Current { get; }  
}
```

- Type `T` can be used only in *output* position; e.g. not as method argument (input)

Contra-variance in interfaces (C# 4)

- When an `I<T>` only consumes/*inputs* `T`'s, it is safe to use an `I<Person>` where an `I<Student>` is expected
- This is contra-variance
- Contra-variance is declared with `in` modifier

```
interface IComparer<in T> {  
    int Compare(T x, T y);  
}
```

- Type `T` can be used only in *input* position; e.g. not as method return type (output)

Variance in function types (C# 4)

- A C# delegate type is
 - co-variant in return type (output)
 - contra-variant in parameters types (input)
- Return type co-variance:

```
Func<int, Student> nthStudent = ...  
Func<int, Person> nthPerson = nthStudent;
```

- Argument type contra-variance:

```
Func<Person, int> personAge = ...  
Func<Student, int> studentAge = personAge;
```

- F# does not support co-variance or contra-variance (yet?)